INITIAL CONCEPT OF OPERATIONS

SWENG 894 Capstone

Chastidy Joanem

1. Concept of Operations

This project is the conclusion to the Penn State Master of Software Engineering program. It is intended to demonstrate the knowledge and skills acquired throughout the program by designing and building a complex, real-world software system. The purpose of this document is to clearly communicate the intent, scope, and initial vision for the proposed project in order to obtain instructor approval.

This Concept of Operations (CONOPS) document serves as an initial planning artifact that defines the mission, problem space, target audience, envisioned features, domain model, architectural approach, technology selections, and project management strategy. While the design will evolve as development progresses, this document establishes a shared understanding of what the system aims to accomplish and how it will be built.

1.1 Approach

The goal of this project is to exercise and integrate technical development, system design, project management, and technical writing skills. Because this is a solo project, maintaining structure, discipline, and transparency is essential.

An Agile-inspired methodology adapted for a team of one will be used. The project will be organized using a Kanban-based workflow, with clearly defined epics and granular tasks tracked as tickets. Backlog refinement will occur regularly, and progress will be evaluated through scheduled self-standups. Development will proceed in three-week sprints.

By the end of the final sprint, a functional Minimum Viable Product (MVP) and supporting documentation will be completed. This project was selected to strengthen skills in greenfield system design, backend and frontend integration, and end-to-end documentation, all of which are critical to the author's professional growth as a software engineer.

1.2 Foundational Assumptions

- This document may evolve or become partially outdated as the project progresses, in accordance with the cone of uncertainty.
- Additional features may be identified and added in future iterations beyond the MVP.

- This document functions as an initial Request for Comments (RFC) rather than a finalized specification.
- As a solo project, all design, development, testing, and documentation responsibilities rest with the author.
- Some tools and frameworks proposed in this document may be new to the author. As a result, certain implementation details may change as experience with these tools increases.

2. Introduction

The project will tentatively be named MicroMatch, a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The system will allow organizations to post small, well-scoped projects and enable students to gain real-world experience without the time commitment of a traditional internship.

2.1 Mission

The mission of this project is to lower the barrier to entry for experiential learning by providing students with access to short-term, skill-focused micro-internships while enabling organizations to efficiently source motivated talent for discrete project-based work.

2.2 Background and Problem Statement

Traditional internships play a significant role in career development, but they present several challenges. Many require long-term commitments, rigid schedules, and competitive application processes that exclude students with limited availability or nontraditional backgrounds. At the same time, startups, nonprofits, and student organizations often have small projects that do not justify hiring a full intern but still require dedicated effort.

As a result, students struggle to gain relevant experience, and organizations struggle to find affordable, short-term help. Existing job boards are optimized for long-term employment rather than small, outcome-driven projects.

A micro-internship marketplace addresses this gap by enabling short-duration, clearly scoped engagements that benefit both parties. Students gain tangible experience and skill validation, while organizations receive focused contributions without long-term obligations.

2.3 Target Audience

Primary Users

- College and university students seeking practical experience
- Early-career professionals building portfolios

Secondary Users

- Startups
- Nonprofit organizations
- Student organizations
- Small business

Additional Stakeholders

- Career services departments
- Faculty advisors
- Program coordinators

2.4 Key Features

The MicroMatch platform will be delivered as a web-based application.

Core features include:

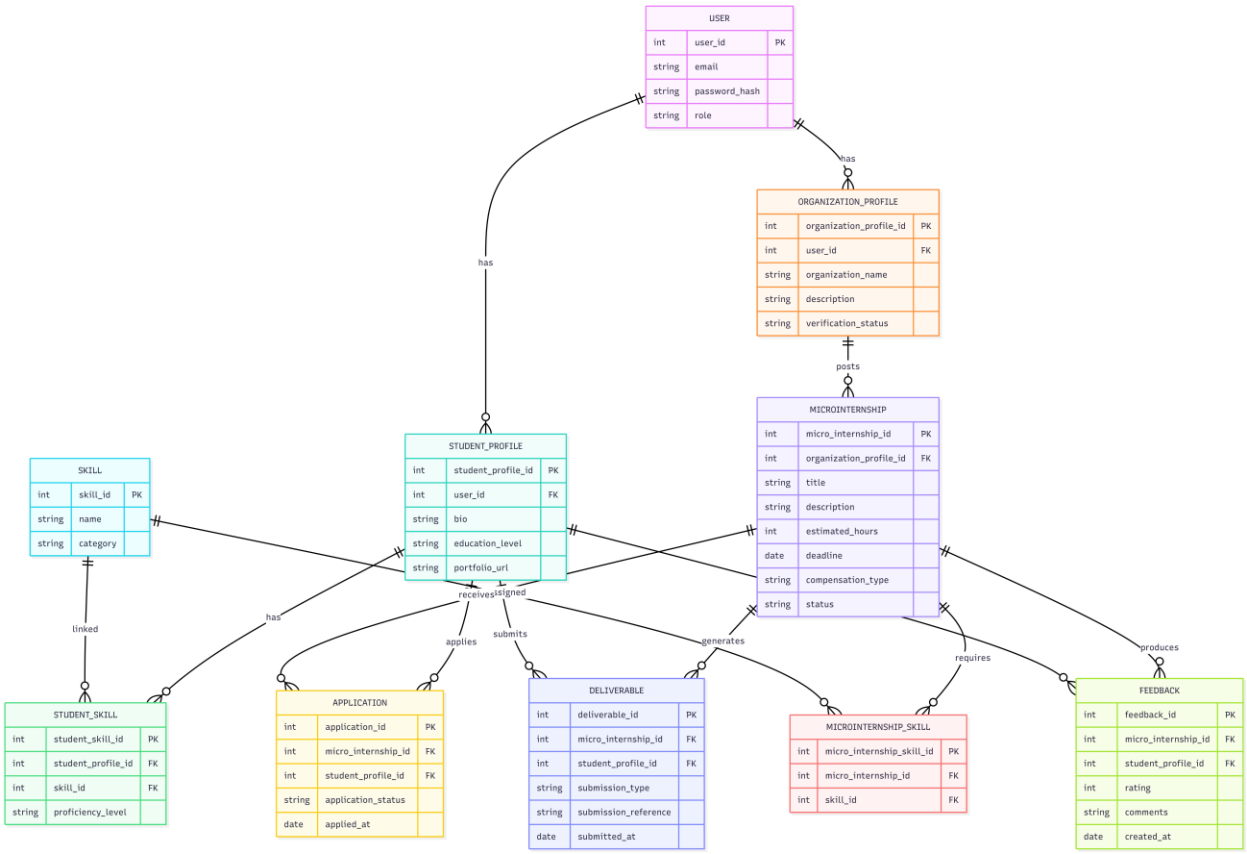
- User account creation with role-based access (Student, Organization, Administrator)
- Student profiles including skills, interests, and past project history
- Organization profiles with verification metadata
- Creation and management of micro-internship listings, including:
    - Required skills
    - Estimated time commitment
    - Deliverables
    - Deadlines
    - Compensation type (paid, unpaid, stipend)
- Application workflow allowing students to apply to projects
- Application status tracking (applied, accepted, rejected, completed)
- Deliverable submission via file upload or external links
- Project completion confirmation
- Feedback and rating system for skill validation
- Administrative moderation and oversight

2.5 Additional Considerations

- Payment processing is out of scope for the MVP and will not be implemented.
- Messaging functionality will be limited to structured comments for status updates.
- The system will emphasize transparency and auditability of project completion.
- Privacy and data protection will be considered when handling user profiles and submissions.

3. Domain Model

The domain model identifies the core entities in the system and their relationships. The model is intended to support an object-oriented design and may evolve during implementation.



Assumptions:

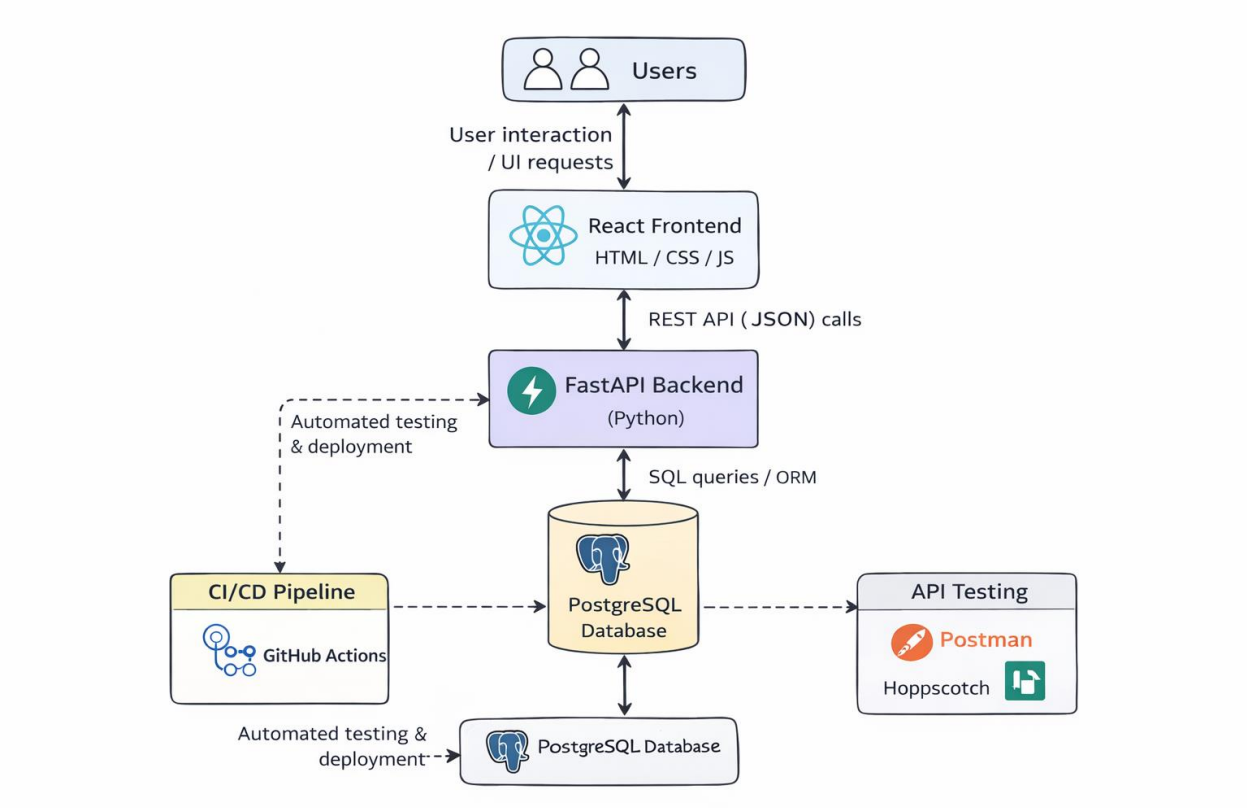
- Using crow's foot notation.
- Subject to change during design and implementation.

Rationale

The domain model represents a micro-internship platform where students and organizations are linked via applications, skills, deliverables, and feedback. It is designed for scalability, clarity, and relational integrity.

3.1 Architecture Diagram

The system will use a React frontend connecting to a FastAPI backend with PostgreSQL as the database. GitHub Actions will manage CI/CD pipelines, and Postman will be used for testing API endpoints. Users will interact via a web browser.



3.2 Programming Languages

Language	Purpose	Rationale
Python	Backend	Widely used, strong ecosystem, supports async FastAPI for REST APIs
FastAPI	Backend Framework	Fast, auto-generated docs, async support, easy testing
React	Frontend Framework	Component-based UI, industry standard, works well with APIs
HTML / CSS / JS	Frontend	Web standard, flexible, integrates with React

PostgreSQL	Database	Powerful relational DB, widely used, supports advanced queries

3.3 Tool Chain

Tool	Type	Rationale
VS Code	IDE	Lightweight, powerful, great extensions for Python, React, and SQL
Git + GitHub	Version control	Industry standard, required for collaboration
GitHub Projects	Kanban board	Integrated with GitHub, organizes tasks, tracks progress
GitHub Actions	CI/CD	Automates tests and deployment, integrates natively with GitHub
Postman / Hoppscotch	API Testing	Test backend endpoints, verify data flows
Jest + React Testing Library	Frontend Testing	Standard for React unit tests
Pytest	Backend Testing	Clean, scalable testing framework for Python
Figma	UI Design	Industry-standard UI design tool, collaborative features
Draw.io / Excalidraw	Diagramming	Easy for architecture, ER, and flow diagrams
Mermaid	UML / ERD visualization	Text-based diagrams for version control and documentation
Microsoft Word / Google Docs	PDF output / formatting	Standard academic documentation
GitHub Wiki / Docs folder	Markdown documentation	Version-controlled docs alongside code

4. Team

Name	Email
Chastidy Joanem	cdj5341@psu.edu

5. Participation

Standard of Work: All deliverables will meet a high standard of quality. Unit test coverage will target a minimum of 90%. Major functional requirements will be validated using integration or end-to-end tests. Accessibility testing will be performed where applicable.

Strategies to Achieve Standard: Agile practices, test-driven development where appropriate, and incremental delivery will be used to ensure quality.

Role Definition: As a solo developer, all roles including requirements analyst, designer, developer, tester, and project manager will be fulfilled by the author.

Deadlines: All deliverables will be submitted by the assigned deadlines, with early submission as a goal where feasible.

6. Completion of Work

Work will be organized through weekly planning, backlog refinement, and continuous documentation. A Kanban board will track all technical and writing tasks. Git branches will be used for feature development, and all changes will be validated through automated tests before merging.

7. Roadblock Mitigation

Potential roadblocks will be identified early during the standups and backlog refinement. Mitigation strategies include targeted research, breaking tasks into smaller units, and seeking guidance from the instructor or external resources.

8. Team Consequences

Not applicable, as this is a solo project.

9. Personal Accountability Statement

I will adhere to the Penn State Code of Conduct and Academic Integrity policies. This project will be completed honestly, independently, and to the best of my ability.

- PSU Code of Conduct: https://studentaffairs.psu.edu/support-safety/conduct/student-code-conduct
- PSU Academic Integrity: https://studentaffairs.psu.edu/support-safety/conduct/academic-misconduct